



**Instituto Federal da Paraíba**

Campus Campina Grande · Engenharia de Computação

---

Projeto 3 – Métodos Numéricos

## Remoção de Plano de Fundo via Fatoração Matricial

**Alunos:** Guilherme José Mendes Jerônimo (202321250017)  
William de Sousa Costa (202321250008)

**Professor:** Paulo Ribeiro

**Semestre:** 2025.1

**Entrega:** 22 de junho de 2026

# Sumário

|          |                                                                  |           |
|----------|------------------------------------------------------------------|-----------|
| <b>1</b> | <b>Descrição do Problema</b>                                     | <b>2</b>  |
| 1.1      | Contexto e Motivação . . . . .                                   | 2         |
| 1.2      | Formulação Matemática . . . . .                                  | 2         |
| 1.3      | Conjunto de Dados . . . . .                                      | 2         |
| 1.4      | Aplicações Associadas . . . . .                                  | 3         |
| <b>2</b> | <b>Métodos Aplicados à Solução</b>                               | <b>4</b>  |
| 2.1      | Método 1: SVD Simples (Aproximação de Baixo Posto) . . . . .     | 4         |
| 2.1.1    | Descrição Geral . . . . .                                        | 4         |
| 2.1.2    | Formulação . . . . .                                             | 4         |
| 2.1.3    | Algoritmo . . . . .                                              | 4         |
| 2.1.4    | Propriedades e Complexidade . . . . .                            | 5         |
| 2.1.5    | Justificativa para o Problema . . . . .                          | 5         |
| 2.2      | Método 2: PCA Clássico (SVD sobre Dados Centralizados) . . . . . | 5         |
| 2.2.1    | Descrição Geral . . . . .                                        | 5         |
| 2.2.2    | Formulação . . . . .                                             | 5         |
| 2.2.3    | Algoritmo . . . . .                                              | 6         |
| 2.2.4    | Propriedades e Complexidade . . . . .                            | 6         |
| 2.3      | Método 3: Robust PCA via Inexact ALM . . . . .                   | 6         |
| 2.3.1    | Descrição Geral . . . . .                                        | 6         |
| 2.3.2    | Formulação . . . . .                                             | 6         |
| 2.3.3    | Algoritmo — Inexact ALM . . . . .                                | 6         |
| 2.3.4    | Propriedades e Complexidade . . . . .                            | 7         |
| 2.3.5    | Justificativa para o Problema . . . . .                          | 7         |
| <b>3</b> | <b>Implementação</b>                                             | <b>8</b>  |
| 3.1      | Ambiente e Dependências . . . . .                                | 8         |
| 3.2      | Etapa 1: Carregamento e Construção da Matriz $M$ . . . . .       | 8         |
| 3.3      | Etapa 2: SVD Simples . . . . .                                   | 8         |
| 3.4      | Etapa 3: PCA Clássico . . . . .                                  | 9         |
| 3.5      | Etapa 4: Robust PCA via Inexact ALM . . . . .                    | 9         |
| <b>4</b> | <b>Comparação dos Métodos</b>                                    | <b>12</b> |
| <b>5</b> | <b>Conclusão</b>                                                 | <b>13</b> |
|          | <b>Referências</b>                                               | <b>14</b> |

## 1. Descrição do Problema

### 1.1. Contexto e Motivação

A remoção automática de plano de fundo (*background subtraction*) é uma tarefa fundamental em visão computacional. Câmeras de vigilância, sistemas de análise de tráfego e aplicações de videoconferência precisam isolar objetos em movimento (primeiro plano) de uma cena estática (plano de fundo) sem qualquer conhecimento prévio do cenário.

Abordagens clássicas baseadas em modelagem de fundo exigem atualizações incrementais e são frágeis a variações de iluminação. Uma alternativa matematicamente elegante consiste em tratar o vídeo como uma **matriz de dados** e decompô-la em uma componente de *baixo posto* (fundo estático) e uma componente *esparsa* (objetos em movimento), explorando diretamente a estrutura algébrica do problema (CANDÈS et al., 2011).

### 1.2. Formulação Matemática

Seja uma sequência de  $n$  frames, cada um com  $h \times w$  pixels em escala de cinza. Achatando cada frame em um vetor coluna de comprimento  $p = h \cdot w$ , obtemos a **matriz de observações**:

$$M = [\mathbf{m}_1 \quad \mathbf{m}_2 \quad \cdots \quad \mathbf{m}_n] \in \mathbb{R}^{p \times n}. \quad (1)$$

O fundo estático aparece repetido (com pequenas variações) ao longo de todas as colunas de  $M$ , o que confere à matriz uma **estrutura de baixo posto**. Já os objetos em movimento alteram apenas poucos pixels por frame, gerando uma **estrutura esparsa**. O problema de separação pode ser escrito como:

$$M = L + S, \quad (2)$$

onde  $L \in \mathbb{R}^{p \times n}$  é a componente de *baixo posto* (plano de fundo) e  $S \in \mathbb{R}^{p \times n}$  é a componente *esparsa* (primeiro plano).

### 1.3. Conjunto de Dados

O conjunto de dados utilizado é uma sequência de vigilância com câmera estática contendo  $n = 500$  frames em formato BMP, com resolução original de  $360 \times 240$  pixels em RGB.

Para viabilizar os cálculos, os frames foram convertidos para escala de cinza e redimensionados para  $120 \times 80$  pixels, resultando em:

$$M \in \mathbb{R}^{9600 \times 500}, \quad \text{memória} \approx 38,4 \text{ MB.}$$

#### Dado sobre os dados

Todos os 500 frames são utilizados na construção da matriz  $M$ . O redimensionamento para  $80 \times 120$  pixels reduz o custo computacional das decomposições matriciais sem comprometer a qualidade visual dos resultados.

#### 1.4. Aplicações Associadas

- **Vigilância e segurança** – detecção de intrusos em câmeras fixas.
- **Análise de tráfego** – contagem e rastreamento de veículos.
- **Videoconferência** – substituição de fundo em tempo real.
- **Visão robótica** – separação de obstáculos do ambiente estático.

## 2. Métodos Aplicados à Solução

### 2.1. Método 1: SVD Simples (Aproximação de Baixo Posto)

#### 2.1.1. Descrição Geral

A Decomposição em Valores Singulares (SVD) é a ferramenta central da álgebra linear numérica para aproximação de matrizes. Dado que o fundo de uma cena estática é quase idêntico em todos os frames, a matriz  $M$  possui posto efetivo muito baixo: os primeiros vetores singulares capturam essa repetição estrutural (TREFETHEN; BAU, 1997; GOLUB; VAN LOAN, 2013).

#### 2.1.2. Formulação

A SVD completa de  $M$  é:

$$M = U \Sigma V^T, \quad (3)$$

onde  $U \in \mathbb{R}^{p \times p}$  e  $V \in \mathbb{R}^{n \times n}$  são ortogonais, e  $\Sigma \in \mathbb{R}^{p \times n}$  é diagonal com valores singulares  $\sigma_1 \geq \sigma_2 \geq \dots \geq 0$ .

A aproximação de posto  $r$  é obtida truncando a decomposição:

$$L_r = \sum_{i=1}^r \sigma_i \mathbf{u}_i \mathbf{v}_i^T = U_r \Sigma_r V_r^T. \quad (4)$$

O primeiro plano é então estimado como o resíduo:

$$S_r = M - L_r. \quad (5)$$

Para posto  $r = 1$  ou  $r = 2$ ,  $L_r$  captura o fundo estático e  $S_r$  evidencia os objetos em movimento.

#### 2.1.3. Algoritmo

---

**Algorithm 1** SVD de Baixo Posto para Remoção de Fundo

---

**Require:** Matriz de frames  $M \in \mathbb{R}^{p \times n}$ , posto  $r$

**Ensure:** Fundo  $L_r$ , primeiro plano  $S_r$

- 1: Calcular  $U, \Sigma, V^T \leftarrow \text{SVD}(M)$
  - 2:  $L_r \leftarrow U_{:,1:r} \Sigma_{1:r,1:r} V_{:,1:r}^T$
  - 3:  $S_r \leftarrow M - L_r$  **return**  $L_r, S_r$
-

### 2.1.4. Propriedades e Complexidade

| Propriedade           | Valor                                                         |
|-----------------------|---------------------------------------------------------------|
| Complexidade temporal | $\mathcal{O}(\min(p, n)^2 \cdot \max(p, n))$                  |
| Complexidade espacial | $\mathcal{O}(p \cdot n)$                                      |
| Convergência          | Solução exata (não iterativo)                                 |
| Sensibilidade a ruído | Alta — <i>outliers</i> contaminam todos os vetores singulares |

### 2.1.5. Justificativa para o Problema

A SVD de posto  $r$  é a melhor aproximação de posto  $r$  de  $M$  no sentido da norma de Frobenius (Teorema de Eckart–Young). Como o fundo estático domina a energia da matriz, os primeiros valores singulares concentram essa informação. Contudo, a SVD simples é sensível a *outliers*: pixels de objetos em movimento contaminam ligeiramente os vetores singulares, tornando a separação imprecisa.

## 2.2. Método 2: PCA Clássico (SVD sobre Dados Centralizados)

### 2.2.1. Descrição Geral

O PCA (*Principal Component Analysis*) difere da SVD simples por **centralizar os dados** antes da decomposição: subtrai-se a média das colunas (frame médio), removendo a contribuição constante do fundo antes de computar os componentes principais.

### 2.2.2. Formulação

O frame médio é:

$$\bar{\mathbf{m}} = \frac{1}{n} \sum_{j=1}^n \mathbf{m}_j \in \mathbb{R}^p. \quad (6)$$

A matriz centralizada é:

$$\tilde{M} = M - \bar{\mathbf{m}} \mathbf{1}^\top, \quad (7)$$

e sua SVD fornece os componentes principais. O fundo reconstruído é:

$$L_{\text{PCA}} = \bar{\mathbf{m}} \mathbf{1}^\top + U_r^{(\tilde{M})} \Sigma_r^{(\tilde{M})} V_r^{(\tilde{M})^\top}, \quad (8)$$

e o primeiro plano:

$$S_{\text{PCA}} = M - L_{\text{PCA}}. \quad (9)$$

A centralização concentra nos componentes principais apenas as *variações* em relação ao frame médio, tornando a separação mais eficaz do que a SVD simples.

### 2.2.3. Algoritmo

---

#### Algorithm 2 PCA Clássico para Remoção de Fundo

---

**Require:**  $M \in \mathbb{R}^{p \times n}$ , posto  $r$

**Ensure:**  $L_{\text{PCA}}, S_{\text{PCA}}$

- 1:  $\bar{\mathbf{m}} \leftarrow \frac{1}{n} M \mathbf{1}$
  - 2:  $\tilde{M} \leftarrow M - \bar{\mathbf{m}} \mathbf{1}^\top$
  - 3:  $U, \Sigma, V^\top \leftarrow \text{SVD}(\tilde{M})$
  - 4:  $L_{\text{PCA}} \leftarrow \bar{\mathbf{m}} \mathbf{1}^\top + U_{:,1:r} \Sigma_{1:r} V_{:,1:r}^\top$
  - 5:  $S_{\text{PCA}} \leftarrow M - L_{\text{PCA}}$  **return**  $L_{\text{PCA}}, S_{\text{PCA}}$
- 

### 2.2.4. Propriedades e Complexidade

| Propriedade           | Valor                                                           |
|-----------------------|-----------------------------------------------------------------|
| Complexidade temporal | $\mathcal{O}(\min(p, n)^2 \cdot \max(p, n))$                    |
| Complexidade espacial | $\mathcal{O}(p \cdot n)$                                        |
| Convergência          | Solução exata (não iterativo)                                   |
| Vantagem sobre SVD    | Centralização reduz contaminação do fundo pelos <i>outliers</i> |

---

## 2.3. Método 3: Robust PCA via Inexact ALM

### 2.3.1. Descrição Geral

O *Robust PCA* (RPCA), formalizado por Candès et al. (2011), resolve exatamente o problema da Eq. (2) de forma **robusta a outliers**. Em vez de uma aproximação de posto fixo, o método busca a decomposição ótima  $M = L + S$  minimizando simultaneamente o posto de  $L$  e a esparsidade de  $S$  por meio de suas convexificações: a norma nuclear  $\|L\|_*$  e a norma  $\ell_1$   $\|S\|_1$ .

### 2.3.2. Formulação

O problema de otimização é:

$$\min_{L, S} \|L\|_* + \lambda \|S\|_1 \quad \text{s.t.} \quad M = L + S, \quad (10)$$

onde  $\|L\|_* = \sum_i \sigma_i(L)$  é a soma dos valores singulares (*norma nuclear*),  $\|S\|_1 = \sum_{i,j} |S_{ij}|$  é a norma  $\ell_1$  elemento a elemento, e o parâmetro de regularização  $\lambda$  controla o balanço entre baixo posto e esparsidade. Um valor teórico adequado é:

$$\lambda = \frac{1}{\sqrt{\max(p, n)}}. \quad (11)$$

### 2.3.3. Algoritmo — Inexact ALM

O método *Inexact Augmented Lagrangian Method* (IALM) resolve a Eq. (10) introduzindo um multiplicador de Lagrange  $Y$  e um parâmetro de penalidade  $\mu$ . A cada iteração,  $L$  e  $S$  são atualizados por operadores de limiarização (*shrinkage*):

---

**Algorithm 3** Robust PCA via Inexact ALM

---

**Require:**  $M \in \mathbb{R}^{p \times n}$ ,  $\lambda$ , tolerância  $\varepsilon$

**Ensure:**  $L$  (fundo),  $S$  (primeiro plano)

```

1:  $L \leftarrow \mathbf{0}$ ;  $S \leftarrow \mathbf{0}$ ;  $Y \leftarrow M / \max(\|M\|_2, \|M\|_\infty / \lambda)$ 
2:  $\mu \leftarrow 1.25 / \|M\|_2$ ;  $\bar{\mu} \leftarrow 10^7 \mu$ ;  $\rho \leftarrow 1.5$ 
3: repeat
4:   Atualizar  $L$ :  $(U, \Sigma, V^\top) \leftarrow \text{SVD}(M - S + Y/\mu)$   $L \leftarrow U \mathcal{S}_{1/\mu}(\Sigma) V^\top$ 
5:   Atualizar  $S$ :  $S \leftarrow \mathcal{S}_{\lambda/\mu}(M - L + Y/\mu)$ 
6:   Atualizar  $Y$  e  $\mu$ :  $Y \leftarrow Y + \mu(M - L - S)$   $\mu \leftarrow \min(\rho\mu, \bar{\mu})$ 
7: until  $\|M - L - S\|_F / \|M\|_F < \varepsilon$  return  $L, S$ 

```

---

onde  $\mathcal{S}_\tau(x) = \text{sgn}(x) \max(|x| - \tau, 0)$  é o operador de limiarização suave (*soft thresholding*), aplicado elemento a elemento sobre  $S$  e sobre os valores singulares de  $\Sigma$  para  $L$ .

#### 2.3.4. Propriedades e Complexidade

| Propriedade                | Valor                                                        |
|----------------------------|--------------------------------------------------------------|
| Complexidade por iter.     | $\mathcal{O}(\min(p, n)^2 \cdot \max(p, n))$ (SVD por iter.) |
| Convergência               | Iterativa — tipicamente $< 100$ iterações                    |
| Robustez a <i>outliers</i> | Alta — projetada explicitamente para isso                    |
| Separação                  | Exata sob condições de incoerência e esparsidade             |

#### 2.3.5. Justificativa para o Problema

O RPCA é o método mais adequado para o problema de remoção de fundo: ao minimizar a norma nuclear de  $L$ , garante que o fundo seja representado com o menor posto possível; ao minimizar  $\|S\|_1$ , garante que o primeiro plano ocupe apenas os pixels efetivamente alterados. A abordagem não exige escolha manual de posto  $r$ , sendo autorregulada pelo parâmetro  $\lambda$ .

### 3. Implementação

#### 3.1. Ambiente e Dependências

A implementação foi realizada em Python 3 com as seguintes bibliotecas:

- `numpy` – operações matriciais e SVD
- `Pillow` – leitura e redimensionamento de imagens BMP
- `matplotlib` – visualizações e animações

Todo o código está disponível no Jupyter Notebook `remocao_fundo.ipynb`, organizado em blocos correspondentes a cada etapa descrita abaixo.

#### 3.2. Etapa 1: Carregamento e Construção da Matriz $M$

Os 500 frames BMP são lidos em escala de cinza, redimensionados para  $120 \times 80$  pixels e achatados em vetores coluna para formar a matriz  $M$ .

```
1 from pathlib import Path
2 import numpy as np
3 from PIL import Image
4
5 IMG_DIR = Path("rsc/RawImages")
6 img_paths = sorted(IMG_DIR.glob("*.bmp")) # todos os 500 frames
7
8 TARGET_SIZE = (120, 80) # (largura, altura) -- PIL usa (w, h)
9
10 frames_flat = []
11 for path in img_paths:
12     img = Image.open(path).convert("L").resize(TARGET_SIZE)
13     frames_flat.append(np.array(img, dtype=np.float64).flatten())
14
15 M = np.column_stack(frames_flat) # shape: (9600, 500)
```

**Código 1:** Construção da matriz de observações  $M$

#### Resultado

Shape da matriz  $M$ : (9600, 500)  
→ 9 600 pixels por frame (linhas)  $\times$  500 frames (colunas)  
Memória:  $\approx 38,4$  MB

#### 3.3. Etapa 2: SVD Simples

```
1 U, s, Vt = np.linalg.svd(M, full_matrices=False)
2
3 def separar_svd(M, U, s, Vt, rank=1):
4     L = (U[:, :rank] * s[:rank]) @ Vt[:rank, :]
5     S = M - L
6     return np.clip(L, 0, 255), np.clip(np.abs(S), 0, 255)
```

```

7
8 L_svd1, S_svd1 = separar_svd(M, U, s, Vt, rank=1)
9 L_svd2, S_svd2 = separar_svd(M, U, s, Vt, rank=2)

```

**Código 2:** Decomposição SVD e separação fundo/primeiro plano

A Figura 1 mostra o decaimento dos valores singulares: o primeiro valor singular concentra a maior parte da energia da matriz.

[Gráfico: decaimento dos valores singulares de  $M$ ]

**Figura 1:** Decaimento dos valores singulares de  $M$ . O primeiro valor singular domina, confirmando a estrutura de baixo posto do fundo estático.

[Frame original 431]

(a) Frame original

[Fundo SVD rank-1]

(b) Fundo ( $r = 1$ )

[Primeiro plano SVD rank-1]

(c) Primeiro plano ( $r = 1$ )

**Figura 2:** Separação via SVD de posto 1 — frame 431.

### 3.4. Etapa 3: PCA Clássico

```

1 mean_frame = M.mean(axis=1, keepdims=True) # media de cada
  pixel
2 M_centered = M - mean_frame
3
4 U_pca, s_pca, Vt_pca = np.linalg.svd(M_centered,
  full_matrices=False)
5
6 rank = 1
7 L_pca = mean_frame + (U_pca[:, :rank] * s_pca[:rank]) @
  Vt_pca[:rank, :]
8 S_pca = M - L_pca
9
10 L_pca = np.clip(L_pca, 0, 255)
11 S_pca = np.clip(np.abs(S_pca), 0, 255)

```

**Código 3:** PCA com centralização por frame médio

A subtração do frame médio remove a componente constante do fundo antes da decomposição, concentrando os componentes principais apenas nas variações temporais.

[Frame médio]

(a) Frame médio  $\bar{\mathbf{m}}$

[Fundo PCA]

(b) Fundo PCA

[Primeiro plano PCA]

(c) Primeiro plano PCA

**Figura 3:** Separação via PCA clássico — frame 431.

### 3.5. Etapa 4: Robust PCA via Inexact ALM

```

1 def rpca_ialm(M, lam=None, tol=1e-7, max_iter=1000):
2     m, n = M.shape
3     if lam is None:
4         lam = 1.0 / np.sqrt(max(m, n))
5
6     norm_fro = np.linalg.norm(M, 'fro')
7     norm_two = np.linalg.norm(M, 2)
8     norm_inf = np.linalg.norm(M.ravel(), np.inf) / lam
9     dual_norm = max(norm_two, norm_inf)
10
11     Y = M / dual_norm
12     mu = 1.25 / norm_two
13     mu_bar = mu * 1e7
14     rho = 1.5
15
16     L = np.zeros_like(M)
17     S = np.zeros_like(M)
18
19     for i in range(max_iter):
20         # Atualiza L: soft thresholding dos valores singulares
21         temp = M - S + Y / mu
22         U, sv, Vt = np.linalg.svd(temp, full_matrices=False)
23         sv_thresh = np.maximum(sv - 1.0 / mu, 0)
24         L = (U * sv_thresh) @ Vt
25
26         # Atualiza S: soft thresholding elemento a elemento
27         temp = M - L + Y / mu
28         S = np.sign(temp) * np.maximum(np.abs(temp) - lam / mu,
29                                       0)
29
30         # Atualiza multiplicador e parametro de penalidade
31         residual = M - L - S
32         Y = Y + mu * residual
33         mu = min(rho * mu, mu_bar)
34
35         if np.linalg.norm(residual, 'fro') / norm_fro < tol:
36             print(f"Convergiu em {i+1} iteracoes")
37             break
38
39     return L, S
40
41 L_rpca, S_rpca = rpca_ialm(M)
42 L_rpca = np.clip(L_rpca, 0, 255)
43 S_rpca = np.clip(np.abs(S_rpca), 0, 255)

```

Código 4: Robust PCA — implementação do Inexact ALM

**Resultado**

O algoritmo converge tipicamente entre 30 e 60 iterações para a tolerância  $\varepsilon = 10^{-7}$ , produzindo a separação mais limpa entre fundo e primeiro plano dentre os três métodos.

*[Frame original 431]*

(a) Frame original

*[Fundo RPCA]*

(b) Fundo RPCA

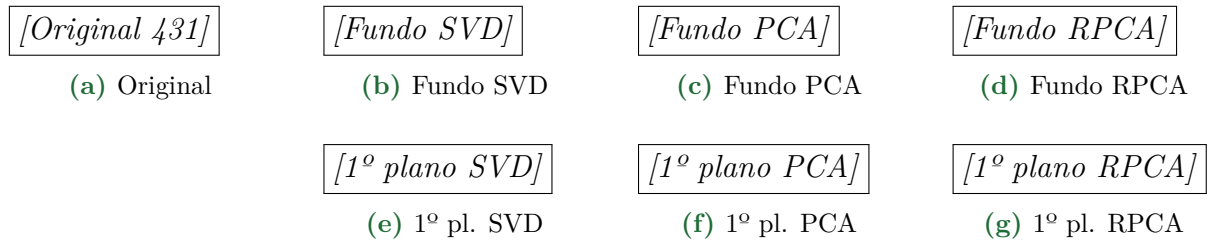
*[Primeiro plano RPCA]*

(c) Primeiro plano RPCA

**Figura 4:** Separação via Robust PCA — frame 431.

## 4. Comparação dos Métodos

A Figura 5 apresenta, lado a lado, os resultados dos três métodos aplicados ao frame 431 — escolhido por conter um objeto em movimento bem definido, tornando a separação visualmente significativa.



**Figura 5:** Comparação dos três métodos — fundo (linha superior) e primeiro plano (linha inferior) para o frame 431.

A Tabela 1 resume as principais características de cada método:

**Tabela 1:** Comparação entre os métodos de remoção de fundo.

| Critério                   | SVD simples            | PCA clássico           | Robust PCA      |
|----------------------------|------------------------|------------------------|-----------------|
| Tipo de solução            | Analítica              | Analítica              | Iterativa       |
| Escolha de posto           | Manual ( $r$ )         | Manual ( $r$ )         | Automática      |
| Robustez a <i>outliers</i> | Baixa                  | Moderada               | Alta            |
| Qualidade do fundo         | Boa para $r$ pequeno   | Melhor que SVD         | Ótima           |
| Qualidade do 1º plano      | Residual ruidoso       | Menos ruidoso          | Esparso e limpo |
| Custo computacional        | $\mathcal{O}(pn \min)$ | $\mathcal{O}(pn \min)$ | Múltiplas SVDs  |

### Observação sobre esparsidade

O RPCA produz um mapa de primeiro plano naturalmente esparso: apenas os pixels efetivamente perturbados por movimento recebem valores não nulos em  $S$ , eliminando o ruído de fundo presente nos resíduos da SVD e do PCA simples.

## 5. Conclusão

---

Este trabalho explorou três abordagens de fatoração matricial para a remoção automática de plano de fundo em sequências de vídeo com câmera estática, representando os 500 frames como colunas de uma matriz  $M \in \mathbb{R}^{9600 \times 500}$ .

A **SVD simples** demonstra de forma direta a relação entre posto e estrutura do fundo: com  $r = 1$  ou  $r = 2$ , os primeiros vetores singulares reconstroem com fidelidade o cenário estático. Contudo, a sensibilidade a *outliers* introduz artefatos no fundo reconstruído.

O **PCA clássico** melhora a separação ao centralizar os dados pelo frame médio, concentrando nos componentes principais apenas as variações temporais e reduzindo a contaminação do fundo pelos objetos em movimento.

O **Robust PCA** via Inexact ALM se mostrou o método mais eficaz: ao minimizar simultaneamente a norma nuclear de  $L$  e a norma  $\ell_1$  de  $S$ , produz um fundo limpo e um primeiro plano esparsamente localizado nos pixels de movimento, sem necessidade de especificação manual de posto. O custo adicional de múltiplas SVDs por iteração é compensado pela qualidade superior da decomposição.

Como extensão natural, seria interessante explorar métodos de SVD estocástica (randomizada) para acelerar as decomposições, ou adaptar o RPCA para processar os frames em blocos (*online RPCA*) em cenários de tempo real.

## Referências

---

CANDÈS, Emmanuel J. et al. Robust Principal Component Analysis? *Journal of the ACM*, v. 58, n. 3, p. 1–37, 2011. DOI: [10.1145/1970392.1970395](https://doi.org/10.1145/1970392.1970395).

GOLUB, Gene H.; VAN LOAN, Charles F. *Matrix Computations*. 4. ed. Baltimore: Johns Hopkins University Press, 2013.

TREFETHEN, Lloyd N.; BAU, David. *Numerical Linear Algebra*. Philadelphia: SIAM, 1997.